

Voting Smart Contract

Code Analysis & Presentation Report

Date	August 2026
Platform	Hedera Testnet
Language	Solidity ^0.8.28
Toolchain	Hardhat 3 + Hedera JS SDK 2.62
Contract	contracts/Voting.sol
Test Suite	test/Voting.t.sol (Foundry / forge-std)

This report analyses the Voting smart contract against the project requirements stated in the course slides (last three pages of *All Lectures.pdf*). It documents every meaningful function and code decision, flags any lines that could attract questions during the live presentation, suggests one small clean-up, and concludes with the exact commands needed to compile, test, and run the demo.

1. Requirements Checklist

The project requirements come from slides 379–381 of *All Lectures.pdf*. Each bullet point is mapped to the corresponding implementation below.

Requirement	Status	Notes
Create a Smart Contract	✓ PASS	contracts/Voting.sol – 310 lines of Solidity
Deploy to a ledger	✓ PASS	deploy-voting.js / demo.js → Hedera Testnet via Hedera JS SDK
Call functions with multiple accounts	✓ PASS	demo.js uses operator, admin, voter1, voter2, voter3
Topics defined by the constructor	✓ PASS	constructor(topicA, topicB, topicC, initialAdmin)
Everybody can vote exactly once	✓ PASS	hasVoted mapping + AlreadyVoted error
Results of voting can be printed	✓ PASS	getResults() returns human-readable string
Contract must be secure	✓ PASS	custom errors, msg.sender guards, no tx.origin, reentrancy-safe pattern
Blocklist (accounts not allowed to vote)	✓ PASS	blocked mapping + admin-only setBlocked / blockVoter / unblockVoter

Result: all 8 requirements are fully satisfied. The contract goes beyond the minimum by adding an auditable, enumerable blocklist, a separated owner/admin role, and a comprehensive 34-test suite.

2. Suggested Changes Before the Presentation

The code is clean and correct. There is **one cosmetic change** worth making to avoid a distraction during the demo: the default topics in `deploy-voting.js` contain a typo that would look odd if a professor reads it aloud.

2.1 Fix the Default Topics Typo (`deploy-voting.js` line 36)

The array `DEFAULT_TOPICS` currently reads:

Current (`deploy-voting.js` : line 36)

```
const DEFAULT_TOPICS = ["Masala Dosai", "Briyani", "SushPani Puri"];
```

"**SushPani Puri**" is clearly a merge of two food names — "Sushi" and "Pani Puri". This is never used in the live demo (`demo.js` hardcodes `["Pizza","Pasta","Sushi"]`), but it will be visible if someone runs `node deploy-voting.js` without flags. Suggested fix — choose three consistent topics:

Suggested fix

```
const DEFAULT_TOPICS = ["Pizza", "Pasta", "Sushi"];
```

Why this matters for the presentation: if a professor or classmate runs `node deploy-voting.js` during the demo, the console will print '*SushPani Puri*' which could invite questions unrelated to the blockchain logic.

2.2 Remove Unused npm Scripts (`package.json` lines 17–18)

The `package.json` references two scripts — `deploy` and `call` — that map to `deploy.js` and `call.js`. Neither of those files exists in the repository. Running them produces an error.

Lines to remove from `package.json`

```
"deploy": "node deploy.js",  
"call": "node call.js"
```

Simply delete those two entries. All other scripts (`compile`, `test`, `demo`, etc.) work correctly.

2.3 No Changes Needed in `Voting.sol` or the Test Suite

The smart contract and all 34 unit tests are correct and presentation-ready as-is. No logic, security, or style changes are recommended.

3. Code Walkthrough — Functions & Key Lines

Below every function and significant code block is explained. Lines marked **[Q]** are the ones most likely to attract a question during the presentation.

3.1 Voting.sol — Contract Header & State Variables

Line(s)	Element	Explanation
1–2	SPDX + pragma	Declares the open-source MIT licence and requires Solidity compiler $\geq$ 0.8.28. The compiler enforces arithmetic overflow checks by default from 0.8.0 onwards.
56	TOPIC_COUNT = 3	A compile-time constant that acts as the fixed array size everywhere. Using a named constant makes the '3' self-documenting and prevents accidental mismatches.
59	address public immutable owner	[Q] <i>immutable</i> means the value is written once in the constructor and baked into the contract bytecode. It cannot be changed after deployment — not even by the owner. Cheaper to read than a regular storage slot.
62	address public admin	Mutable; can be handed over with setAdmin(). Separated from owner so the power to exclude voters sits in exactly one place and can be transferred without redeploying.
65	string[TOPIC_COUNT] private _topics	Fixed-size array of three strings. Private means it cannot be read directly from outside the contract; getTopic() is the public accessor.
68	uint256[TOPIC_COUNT] private _tally	One counter per topic. Incremented inside an unchecked block (see vote()).
71	mapping(address => bool) public hasVoted	[Q] A mapping from wallet address to boolean. Public auto-generates a getter. Set to true the moment a vote is cast — this is the 'exactly once' enforcement.
74	mapping(address => bool) public blocked	O(1) lookup: is this address on the blacklist? Public so anyone can audit it.
77	address[] private _blockedList	Parallel array that lets anyone enumerate all blocked accounts. Required because a mapping alone is not iterable on-chain.
80	mapping(address => uint256) private _blockedSlot	Stores the 1-based index of each address in _blockedList. Enables O(1) removal via the swap-and-pop pattern. 0 means 'not in the list'.
83	uint256 public totalVotes	Running total across all topics. Useful for a quick sanity check without summing the tally array.

3.2 constructor() — Lines 89–114

```
constructor(
    string memory topicA,
    string memory topicB,
    string memory topicC,
    address initialAdmin
) {
    if (bytes(topicA).length == 0 || ...) revert EmptyTopic();
    if (initialAdmin == address(0)) revert ZeroAddress();
    owner = msg.sender;
    admin = initialAdmin;
    _topics[0..2] = topicA, topicB, topicC;
    emit AdminChanged(address(0), initialAdmin);
}
```

[Q] Why bytes(topicA).length? In Solidity, you cannot compare a string directly to "". Casting to bytes exposes the length. An empty string would mean a ballot option with no label — a permanent deployment mistake since the topics can never change after construction.

[Q] Why is initialAdmin required to be non-zero? address(0) is the burn/null address; no private key controls it, so a zero admin would lock the blacklist permanently with no way to add or remove entries.

3.3 Modifiers — Lines 123–131

```
modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

modifier onlyAdmin() {
    if (msg.sender != admin) revert NotAdmin();
    _;
}
```

[Q] Why msg.sender and not tx.origin? tx.origin is the original wallet that started a transaction chain. If the owner ever calls into a malicious contract that then calls setAdmin(), tx.origin would still be the owner and the attack would succeed. msg.sender is the *immediate* caller, so this attack is impossible.

3.4 vote() — Lines 141–158

```
function vote(uint256 topicIndex) external {
    if (topicIndex >= TOPIC_COUNT) revert InvalidTopic(topicIndex);
    if (blocked[msg.sender]) revert AccountBlocked();
    if (hasVoted[msg.sender]) revert AlreadyVoted();

    hasVoted[msg.sender] = true; // Effects first
    unchecked {
        _tally[topicIndex] += 1;
        totalVotes += 1;
    }
    emit VoteCast(msg.sender, topicIndex);
}
```

[Q] Check order — why blocked before hasVoted? A blocked address gets a clearer error message (*AccountBlocked*) rather than *AlreadyVoted*, which would be confusing if the address had voted before being blocked.

[Q] Why unchecked? The number of distinct Ethereum/Hedera addresses (2^{160}) is far larger than `uint256 max` ($2^{256} - 1$), so these counters can never physically overflow. The unchecked block saves a small amount of gas by skipping Solidity's built-in overflow guard.

[Q] Effects before interactions (CEI pattern)? `hasVoted` is set *before* the event is emitted. There are no external calls, so re-entrancy is not reachable here, but the pattern is maintained as a best practice in case the contract is ever extended.

3.5 `setBlocked()` — Lines 168–199

```
function setBlocked(address account, bool isBlocked) public onlyAdmin {
    if (account == address(0)) revert ZeroAddress();
    if (account == owner)      revert OwnerCannotBeBlocked();
    if (account == admin)     revert AdminCannotBeBlocked();
    if (blocked[account] == isBlocked) return; // idempotent

    blocked[account] = isBlocked;
    if (isBlocked) {
        _blockedList.push(account);
        _blockedSlot[account] = _blockedList.length; // 1-based
    } else {
        // Swap-and-pop: O(1) removal
        uint256 slot = _blockedSlot[account];
        address moved = _blockedList[_blockedList.length - 1];
        _blockedList[slot - 1] = moved;
        _blockedSlot[moved] = slot;
        _blockedList.pop();
        _blockedSlot[account] = 0;
    }
    emit AccountBlockedSet(account, isBlocked);
}
```

[Q] Why can't the admin block the owner or itself? Blocking the owner would let a rogue admin lock out the deployer. Blocking itself would leave the list unmanageable (the admin could never unblock anyone either).

[Q] Idempotent early return? If you try to block an already-blocked address, the function returns early without modifying storage or emitting an event. This prevents duplicate entries in `_blockedList` and saves gas.

[Q] Swap-and-pop? Removing an element from the middle of an array by shifting all elements right costs $O(n)$ gas. Swap-and-pop swaps the target element with the last element and removes the last element — always $O(1)$. Order in the list is not meaningful (it is just an audit trail), so this is safe.

[Q] Why `_blockedSlot` stores `index+1`? A mapping returns 0 for any key that was never set. Storing 1-based indices means 0 unambiguously means 'not in the list', preventing a bug where slot 0 could be confused with an absent entry.

3.6 `blockVoter()` / `unblockVoter()` — Lines 202–209

```
function blockVoter(address account) external { setBlocked(account, true); }
function unblockVoter(address account) external { setBlocked(account, false); }
```

Thin wrappers that let callers use a readable function name instead of passing a boolean flag. They inherit the onlyAdmin guard through the delegation to setBlocked().

3.7 setAdmin() — Lines 212–216

```
function setAdmin(address newAdmin) external onlyOwner {
    if (newAdmin == address(0)) revert ZeroAddress();
    emit AdminChanged(admin, newAdmin);
    admin = newAdmin;
}
```

[Q] Why emit before assignment? The event captures the *previous* admin address. Emitting first makes the log contain the old value; assigning first would lose it. There is no external call here, so re-entrancy is not a concern.

3.8 Blocklist Readers — Lines 223–242

```
getBlockedCount() → uint256          // number of blocked addresses
getBlockedVoter(uint256 index) → address // address at position i
getBlockedVoters() → address[] memory // whole list in one call
```

Three read-only views. getBlockedVoters() returns an unbounded array which is fine because only the admin can grow the list. The scripts prefer the getBlockedCount + getBlockedVoter(i) pair because the Hedera SDK cannot decode a dynamic address[] return value conveniently.

3.9 getResults() — Lines 253–260

```
function getResults() external view returns (string memory) {
    return string.concat(
        _topics[0], ":", _toString(_tally[0]), " | ",
        _topics[1], ":", _toString(_tally[1]), " | ",
        _topics[2], ":", _toString(_tally[2])
    );
}
// Example output: "Pizza: 2 | Pasta: 1 | Sushi: 0"
```

[Q] Why a single string instead of an array? The Hedera JS SDK's ContractCallQuery can decode a single string return with getString(0). Decoding a string[] would require custom ABI parsing. Returning one formatted string keeps the off-chain code simpler.

3.10 Utility Views — Lines 263–286

```
getTopic(uint256 topicIndex) → string    // label of topic 0, 1 or 2
getVotes(uint256 topicIndex) → uint256   // vote count for a topic
canVote(address account) → bool          // !blocked && !hasVoted
whoAmI() → address                      // returns msg.sender
```

[Q] What is whoAmI() for? Hedera maps every native account (0.0.xxxx) to an EVM address. This helper lets you call the contract and see exactly which EVM address the network associates with your key, so you know which address to pass to blockVoter().

3.11 _toString() — Lines 293–308 (internal helper)

```
function _toString(uint256 value) private pure returns (string memory) {
```

```

    if (value == 0) return "0";
    // Count digits
    uint256 digits;
    for (uint256 temp = value; temp != 0; temp /= 10) digits++;
    // Fill buffer right-to-left
    bytes memory buffer = new bytes(digits);
    while (value != 0) {
        digits--;
        buffer[digits] = bytes1(uint8(48 + (value % 10)));
        value /= 10;
    }
    return string(buffer);
}

```

[Q] Why not use OpenZeppelin's Strings library? This avoids an external dependency. The logic is only 15 lines, and for a course project it is clearer to show the implementation explicitly. The algorithm is standard: count digits in the first loop, then fill a byte buffer right-to-left using ASCII offset 48 (the character '0').

48 + value % 10 — 48 is the ASCII code of '0'. Adding the digit (0–9) gives '0'–'9'.

4. Off-chain Scripts Explained

create-accounts.js

Creates VOTER1...VOTER3 (and ADMIN) Hedera Testnet accounts and writes their IDs and ECDSA private keys into .env. ECDSA keys are required because only ECDSA accounts have a derived EVM address that becomes msg.sender on-chain. ED25519 accounts would resolve to Hedera's 'long-zero' address form, which the contract cannot distinguish per voter.

deploy-voting.js

Compiles the contract artifact (already compiled by Hardhat), reads it, and deploys via ContractCreateFlow. Constructor arguments (three topic strings + admin address) are encoded with ContractFunctionParameters. After deployment the contract ID is written back into .env so other scripts pick it up automatically.

demo.js

End-to-end live demo: deploys a fresh contract, runs the 8-step story (block voter3, three successful votes, three rejections, print blocklist and tally). Uses five separate clients (one per account) so each transaction is signed by the correct key. Prints checkmarks (■) and crosses (■) for each step.

vote.js

Cast a single vote from a named account. First calls canVote() and getTopic() to print a pre-flight status, then sends the vote() transaction. On rejection it decodes the custom error name from the 4-byte selector and prints it (e.g. *AlreadyVoted()*).

block.js

Admin-only: block or unblock an account. Accepts a label (voter3) or raw 0x address. Calls setBlocked(address, bool). Demonstrates the NotAdmin error when a non-admin account sends the transaction.

status.js

Read-only dashboard: calls owner, admin, getResults(), getTopic/getVotes (per topic), getBlockedCount/getBlockedVoter, and hasVoted/blocked/canVote (per account). No state is changed. Safe to run as many times as needed during the demo.

lib/hedera.js

Shared utilities: parsePrivateKey() — handles both raw 64-hex ECDSA keys and DER-encoded keys without silently returning the wrong key type; loadAccount() — reads ID+KEY from .env; loadVoters() — iterates VOTER1...VOTERn; makeClient() — Testnet client with a 20 HBAR max fee cap; loadErrorSelectors() — builds a 4-byte-selector→name map from the ABI so reverts are reported by name;

`fetchRevertReason()` — re-queries the record with receipt validation off to extract the revert data;
`updateEnvFile()` — idempotently writes key=value pairs into `.env`.

5. Commands to Run the Code

5.1 Prerequisites

Node.js ≥ 18	<code>node --version</code>
npm packages	<code>npm install</code>
.env file	Copy <code>.env.example</code> to <code>.env</code> and fill in <code>HEDERA_OPERATOR_ID</code> and <code>HEDERA_OPERATOR_KEY</code>
Hedera account	Free Testnet account at https://portal.hedera.com/

5.2 Step-by-step Workflow

Step 1 — Install dependencies

```
npm install
```

Downloads `@hashgraph/sdk`, `dotenv`, `js-sha3`, `hardhat`, `forge-std` into `node_modules`.

Step 2 — Compile the contract

```
npm run compile  
# or: npx hardhat compile
```

Hardhat compiles `contracts/Voting.sol` and writes the ABI + bytecode to `artifacts/contracts/Voting.sol/Voting.json`. Required before any deploy step.

Step 3 — Run the unit tests

```
npm test  
# or: npx hardhat test solidity
```

Runs all 34 Foundry/forge-std tests inside Hardhat's in-process EVM. All tests should pass in < 2 seconds. No Hedera account or HBAR needed.

Step 4 — Create accounts

```
node create-accounts.js  
# Creates VOTER1, VOTER2, VOTER3 (20 HBAR each)  
  
node create-accounts.js --prefix ADMIN --count 1  
# Creates ADMIN (20 HBAR)
```

Creates ECDSA accounts on Hedera Testnet funded from the operator. Writes `VOTER1_ID`, `VOTER1_KEY`, ... and `ADMIN_ID`, `ADMIN_KEY` into `.env`.

Step 5a — Full automated demo (recommended for presentation)

```
node demo.js
```

Deploys a fresh contract and runs all 8 steps automatically. Prints `■ / ■` for every action. Topics are Pizza, Pasta, Sushi.

Step 5b — Manual deployment

```
node deploy-voting.js
```

```
# or with custom topics:
node deploy-voting.js --topics "Tea,Coffee,Water"
# or with a different admin:
node deploy-voting.js --admin voter1
```

Deploys the contract and saves CONTRACT_ID to .env.

Step 6 — Check current state

```
node status.js
```

Read-only: prints owner, admin, tally per topic, blocklist, and per-account flags. Run this between other commands to verify state changes.

Step 7 — Cast a vote

```
node vote.js --as voter1 --topic 0
node vote.js --as voter2 --topic 1
node vote.js --as voter3 --topic 0    # will fail if voter3 is blocked
```

--as: account label (operator, voter1, voter2, voter3) --topic: 0, 1 or 2

Step 8 — Block / unblock a voter

```
node block.js --account voter3          # sent by admin (default)
node block.js --account voter3 --unblock
node block.js --account voter1 --as voter2  # demonstrates NotAdmin error
node block.js --account voter1 --as operator # owner is not the admin
```

Sent by the admin by default. Use --as to demonstrate rejection.

5.3 Quick Reference Card

Command	Purpose
npm install	Install all dependencies
npm run compile	Compile Voting.sol → artifact JSON
npm test	Run 34 unit tests (no Hedera needed)
node create-accounts.js	Create voter1/2/3 accounts on Testnet
node create-accounts.js --prefix ADMIN --count 1	Create admin account
node demo.js	Full end-to-end automated demo
node deploy-voting.js	Deploy contract, save CONTRACT_ID to .env
node status.js	Print current contract state
node vote.js --as voter1 --topic 0	voter1 votes for topic 0 (Pizza)
node block.js --account voter3	Admin blocks voter3
node block.js --account voter3 --unblock	Admin unblocks voter3

5.4 Recommended Live Demo Order

- **1.** npm run compile — show the Solidity file compiles cleanly
- **2.** npm test — all 34 tests pass instantly
- **3.** node status.js — show the initial state (empty tally, no blocklist)
- **4.** node demo.js — run the full story with / output
- **5.** node status.js — show the final state (Pizza:2, Pasta:1, voter3 blocked)